

LAB 007

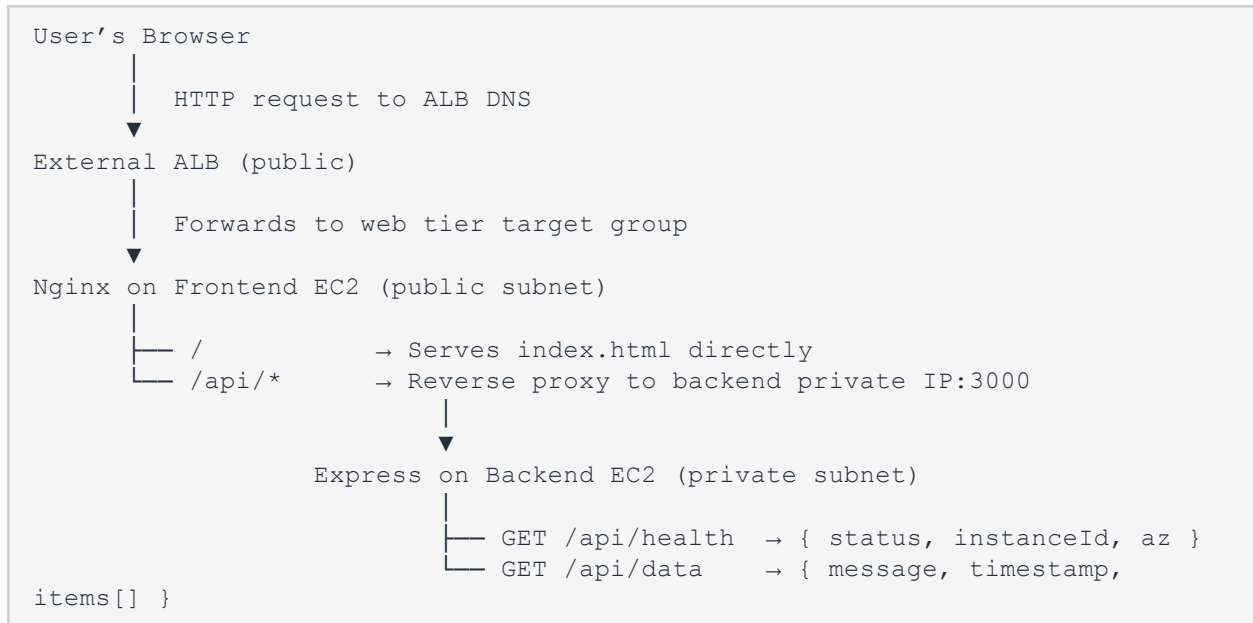
Application Code & Setup

How the Application Works

Your two-tier architecture runs a real application with a frontend and backend that communicate through the network:

Component	Description
Frontend (Public Tier)	Nginx serves an HTML page. When the page loads, JavaScript calls <code>/api/health</code> and <code>/api/data</code> . Nginx reverse-proxies any request starting with <code>/api/</code> to the backend.
Backend (Private Tier)	A Node.js Express server running on port 3000. It exposes two endpoints: <code>GET /api/health</code> and <code>GET /api/data</code> . It is NOT accessible from the internet — only from the frontend tier.
Reverse Proxy	Nginx on the frontend catches <code>/api/*</code> requests and forwards them to the backend's private IP on port 3000. The browser never talks to the backend directly.

Request Flow



Backend: Node.js Express API

This is a single file: `server.js`. It runs on port 3000 and exposes two endpoints.

server.js

```

const express = require("express");
const { execSync } = require("child_process");

const app = express();
const PORT = 3000;

// Helper: get instance metadata (IMDSv2)
function getMeta(path) {
  try {
    const token = execSync(
      'curl -s -X PUT "http://169.254.169.254/latest/api/token" ' +
      '-H "X-aws-ec2-metadata-token-ttl-seconds: 21600"'
    ).toString().trim();
    return execSync(
      `curl -s -H "X-aws-ec2-metadata-token: ${token}" ` +

```

```

        `http://169.254.169.254/latest/meta-data/${path}`
    ).toString().trim();
  } catch (e) {
    return "unknown";
  }
}

// GET /api/health
app.get("/api/health", (req, res) => {
  res.json({
    status: "healthy",
    tier: "backend",
    instanceId: getMeta("instance-id"),
    availabilityZone: getMeta("placement/availability-zone"),
    timestamp: new Date().toISOString(),
  });
});

// GET /api/data
app.get("/api/data", (req, res) => {
  res.json({
    message: "Hello from the Backend Tier!",
    instanceId: getMeta("instance-id"),
    availabilityZone: getMeta("placement/availability-zone"),
    timestamp: new Date().toISOString(),
    items: [
      { id: 1, name: "VPC", description: "Virtual Private Cloud" },
      { id: 2, name: "ALB", description: "Application Load Balancer" },
      { id: 3, name: "ASG", description: "Auto Scaling Group" },
      { id: 4, name: "IAM", description: "Identity and Access Management" },
      { id: 5, name: "SSM", description: "Systems Manager Parameter Store" }
    ],
  });
});

app.listen(PORT, () => {
  console.log(`Backend API running on port ${PORT}`);
});

```

Frontend: HTML + JavaScript

This is a single file: `index.html`. Nginx serves it. On page load, it calls `/api/health` and `/api/data` through the Nginx reverse proxy.

index.html

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Two-Tier App</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; background: #f0f2f5; color: #333;
padding: 40px 20px; }
    .container { max-width: 800px; margin: 0 auto; }
    h1 { color: #232F3E; margin-bottom: 8px; }
    .subtitle { color: #666; margin-bottom: 30px; }
    .card { background: white; border-radius: 8px; padding: 24px;
margin-bottom: 20px; box-shadow: 0 1px 3px rgba(0,0,0,0.1); }
    .card h2 { font-size: 18px; color: #1A73E8; margin-bottom: 12px; }
    .status { display: inline-block; padding: 4px 12px; border-radius: 12px;
font-size: 14px; font-weight: bold; }
    .status.healthy { background: #E8F5E9; color: #1B7D40; }
    .status.error { background: #FFEBEE; color: #C62828; }
    .meta { color: #666; font-size: 14px; margin-top: 8px; }
    table { width: 100%; border-collapse: collapse; margin-top: 12px; }
    th { background: #232F3E; color: white; padding: 10px 12px; text-align:
left; }
    td { padding: 10px 12px; border-bottom: 1px solid #eee; }
    tr:hover { background: #f5f5f5; }
    .loading { color: #999; font-style: italic; }
  </style>
</head>
<body>
  <div class="container">
    <!-- CHANGE THE NAME BELOW TO YOUR FULL NAME -->
    <h1>Two-Tier App &mdash; [YOUR NAME HERE]</h1>
    <p class="subtitle">Web Tier &rarr; Nginx Reverse Proxy &rarr; Backend
API</p>

    <div class="card">
      <h2>Backend Health Check</h2>
      <div id="health"><span class="loading">Checking
backend...</span></div>
    </div>
```

```

<div class="card">
  <h2>Backend Data</h2>
  <div id="data"><span class="loading">Fetching data...</span></div>
</div>
</div>

<script>
  fetch("/api/health")
    .then(r => r.json())
    .then(d => {
      document.getElementById("health").innerHTML = `
        <span class="status healthy">${d.status}</span>
        <div class="meta">
          Instance: ${d.instanceId}<br>
          AZ: ${d.availabilityZone}<br>
          Time: ${d.timestamp}
        </div>`;
    })
    .catch(() => {
      document.getElementById("health").innerHTML =
        '<span class="status error">unreachable</span>';
    });

  fetch("/api/data")
    .then(r => r.json())
    .then(d => {
      let rows = d.items.map(i =>
        `<tr><td>${i.id}</td><td>${i.name}</td><td>${i.description}</td></tr>`
      ).join("");
      document.getElementById("data").innerHTML = `
        <p>${d.message}</p>
        <div class="meta">Instance: ${d.instanceId} | AZ:
        ${d.availabilityZone}</div>
        <table>
          <tr><th>ID</th><th>Name</th><th>Description</th></tr>
          ${rows}
        </table>`;
    })
    .catch(() => {
      document.getElementById("data").innerHTML =
        '<span class="status error">Failed to load data</span>';
    });
</script>
</body>
</html>

```

⚠ Warning

You MUST change [YOUR NAME HERE] in the HTML to your full name before deploying. Your name must be visible in the browser for submission.

User Data Scripts

These scripts go in your `userdata/` folder and are referenced by your Terraform launch templates. They install everything and start the application automatically when the EC2 instance boots.

userdata/web.sh (Frontend Tier)

This script installs Nginx, places the `index.html`, and configures the reverse proxy to forward `/api/*` requests to the backend.

```
#!/bin/bash
yum update -y
yum install -y nginx

# — Create the frontend HTML —
cat > /usr/share/nginx/html/index.html << 'HTMLEOF'
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Two-Tier App</title>
  <style>
    * { margin: 0; padding: 0; box-sizing: border-box; }
    body { font-family: Arial, sans-serif; background: #f0f2f5;
           color: #333; padding: 40px 20px; }
    .container { max-width: 800px; margin: 0 auto; }
    h1 { color: #232F3E; margin-bottom: 8px; }
    .subtitle { color: #666; margin-bottom: 30px; }
    .card { background: white; border-radius: 8px; padding: 24px;
           margin-bottom: 20px; box-shadow: 0 1px 3px rgba(0,0,0,0.1); }
    .card h2 { font-size: 18px; color: #1A73E8; margin-bottom: 12px; }
    .status { display: inline-block; padding: 4px 12px; border-radius: 12px;
             font-size: 14px; font-weight: bold; }
    .status.healthy { background: #E8F5E9; color: #1B7D40; }
    .status.error { background: #FFEBEE; color: #C62828; }
    .meta { color: #666; font-size: 14px; margin-top: 8px; }
    table { width: 100%; border-collapse: collapse; margin-top: 12px; }
    th { background: #232F3E; color: white; padding: 10px 12px;
         text-align: left; }
    td { padding: 10px 12px; border-bottom: 1px solid #eee; }
    tr:hover { background: #f5f5f5; }
    .loading { color: #999; font-style: italic; }
  </style>
</head>
<body>
```

```

<div class="container">
  <!-- CHANGE THE NAME BELOW TO YOUR FULL NAME -->
  <h1>Two-Tier App &mdash; [YOUR NAME HERE]</h1>
  <p class="subtitle">Web Tier &rarr; Nginx Reverse Proxy &rarr; Backend
API</p>
  <div class="card">
    <h2>Backend Health Check</h2>
    <div id="health"><span class="loading">Checking
backend...</span></div>
  </div>
  <div class="card">
    <h2>Backend Data</h2>
    <div id="data"><span class="loading">Fetching data...</span></div>
  </div>
</div>
<script>
  fetch("/api/health")
    .then(r => r.json())
    .then(d => {
      document.getElementById("health").innerHTML = `
        <span class="status healthy">${d.status}</span>
        <div class="meta">
          Instance: ${d.instanceId}<br>
          AZ: ${d.availabilityZone}<br>
          Time: ${d.timestamp}
        </div>`;
    })
    .catch(() => {
      document.getElementById("health").innerHTML =
        '<span class="status error">unreachable</span>';
    });
  fetch("/api/data")
    .then(r => r.json())
    .then(d => {
      let rows = d.items.map(i =>
        `<tr><td>${i.id}</td><td>${i.name}</td>
        <td>${i.description}</td></tr>`
      ).join("");
      document.getElementById("data").innerHTML = `
        <p>${d.message}</p>
        <div class="meta">
          Instance: ${d.instanceId} | AZ: ${d.availabilityZone}</div>
        <table>
          <tr><th>ID</th><th>Name</th><th>Description</th></tr>
          ${rows}
        </table>`;
    })
    .catch(() => {
      document.getElementById("data").innerHTML =
        '<span class="status error">Failed to load data</span>';
    });
</script>
</body>
</html>

```

```

HTMLEOF

# —— Configure Nginx reverse proxy ——
# Replace BACKEND_IP below with one of your backend instance's private IP
cat > /etc/nginx/conf.d/reverse-proxy.conf << 'NGINXEOF'
server {
    listen 80;
    server_name _;

    root /usr/share/nginx/html;
    index index.html;

    location / {
        try_files $uri $uri/ =404;
    }

    location /api/ {
        proxy_pass http://BACKEND_IP:3000;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
NGINXEOF

# Remove the default server block to avoid conflicts
rm -f /etc/nginx/conf.d/default.conf
sed -i '/^\s*server\s*{/,/^\s*}/d' /etc/nginx/nginx.conf 2>/dev/null || true

# Start Nginx
systemctl start nginx
systemctl enable nginx

```

⚠ Warning

You must replace `BACKEND_IP` in the script with the actual private IP of one of your backend instances.

You can either hardcode it after your first deploy, or use a Terraform variable to inject it dynamically.

★ Bonus

Instead of hardcoding a single backend IP, create an Internal ALB in your private subnets that load balances across your backend ASG instances.

Then replace `BACKEND_IP:3000` with your Internal ALB's DNS name:80 in the Nginx config. This is how production two-tier architectures work — the frontend never talks to individual backend instances directly.

userdata/backend.sh (Backend Tier)

This script installs Node.js, creates the Express API, and starts it on port 3000.

```
#!/bin/bash
yum update -y

# — Install Node.js 18 —
curl -fsSL https://rpm.nodesource.com/setup_18.x | bash -
yum install -y nodejs

# — Create app directory —
mkdir -p /home/ec2-user/app
cd /home/ec2-user/app

# — Initialize and install Express —
npm init -y
npm install express

# — Create server.js —
cat > server.js << 'APPEOF'
const express = require("express");
const { execSync } = require("child_process");

const app = express();
const PORT = 3000;

function getMeta(path) {
  try {
    const token = execSync(
      'curl -s -X PUT "http://169.254.169.254/latest/api/token" ' +
      '-H "X-aws-ec2-metadata-token-ttl-seconds: 21600"'
    ).toString().trim();
    return execSync(
      `curl -s -H "X-aws-ec2-metadata-token: ${token}" ` +
      `http://169.254.169.254/latest/meta-data/${path}`
    ).toString().trim();
  } catch (e) {
    return "unknown";
  }
}

app.get("/api/health", (req, res) => {
  res.json({
    status: "healthy",
    tier: "backend",
    instanceId: getMeta("instance-id"),
    availabilityZone: getMeta("placement/availability-zone"),
    timestamp: new Date().toISOString(),
  });
});
```

```
});

app.get("/api/data", (req, res) => {
  res.json({
    message: "Hello from the Backend Tier!",
    instanceId: getMeta("instance-id"),
    availabilityZone: getMeta("placement/availability-zone"),
    timestamp: new Date().toISOString(),
    items: [
      { id: 1, name: "VPC", description: "Virtual Private Cloud" },
      { id: 2, name: "ALB", description: "Application Load Balancer" },
      { id: 3, name: "ASG", description: "Auto Scaling Group" },
      { id: 4, name: "IAM", description: "Identity and Access Management" },
      { id: 5, name: "SSM", description: "Systems Manager Parameter Store" }
    ],
  });
});

app.listen(PORT, () => {
  console.log(`Backend API running on port ${PORT}`);
});
APPEOF

# — Run the server in the background —
chown -R ec2-user:ec2-user /home/ec2-user/app
node server.js &
```

How to Verify It Works

After deploying with Terraform, follow these steps to verify the full flow:

1. Test the Frontend via ALB

Open your browser and go to the External ALB DNS name. You should see the Two-Tier App page with your name. The health check card should show "healthy" with the backend instance ID, and the data card should display the table of AWS services.

2. Test the Backend Directly (from Bastion)

SSH into your bastion host, then SSH into a backend instance and test the API locally:

```
# From your machine → bastion
ssh -i your-key.pem ec2-user@<BASTION-PUBLIC-IP>

# From bastion → backend instance
ssh -i your-key.pem ec2-user@<BACKEND-PRIVATE-IP>

# On the backend instance, test the API
curl http://localhost:3000/api/health
curl http://localhost:3000/api/data
```

You should see JSON responses with the instance ID and availability zone.

3. Confirm the Backend is NOT Public

Try accessing a backend instance's private IP directly from your browser or local machine. It should **NOT** work — the backend is only reachable through the Nginx reverse proxy or via the bastion. This confirms your security group and subnet configuration is correct.

File Overview

Here is where each file goes in your project:

```
terraform-challenge/
├── main.tf
├── network.tf
├── security.tf
├── compute.tf
├── alb.tf
├── secrets.tf
├── variables.tf
├── outputs.tf
├── terraform.tfvars      ← DO NOT push to GitHub
├── .gitignore
└── userdata/
    ├── web.sh            ← Nginx + index.html + reverse proxy
    └── backend.sh        ← Node.js + Express API
```



Note

The user data scripts contain the entire application. You do not need separate app files — everything is embedded in the shell scripts and gets created on the EC2 instances at boot time.

The index.html and server.js sections above are shown separately for readability, but they are already included inside the user data scripts.

Important Notes

- **Replace your name:** Change [YOUR NAME HERE] in the web.sh user data script before deploying.
- **Replace BACKEND_IP:** After your first terraform apply, get a backend instance's private IP from the console or outputs, update web.sh, and re-deploy the web tier.
- **Backend SG port:** Your Backend-SG must allow inbound TCP port 3000 from Web-SG (not port 80). The Express server listens on 3000.
- **NAT Gateway required:** The backend instances need internet access (via NAT Gateway) to install Node.js and npm packages during boot.
- **Boot time:** The backend takes 1–2 minutes to boot because it installs Node.js and npm packages. If the frontend shows "unreachable" at first, wait and refresh.

- **ALB health check:** Set the web tier target group health check to path / on port 80. For an internal ALB (bonus), set it to path /api/health on port 3000.

This document is a companion to Lab 007. Refer to the main lab document for architecture requirements and submission instructions.